

Report: Deep Bluffing

Akshat Pandey

1 Introduction

In case of game theory, Texas Hold'em poker has challenges because of its imperfect information and probabilistic nature. Here, we developed a competitive poker agent who is designed to make autonomous and mathematically sound decisions. The core objective was to build a system that not only adheres to the strict rules of poker but also employs a rational decision-making framework to maximize performance.

2 System Architecture

So as to separate the game's rules from the strategic reasoning, the code is divided into two separate segments:

- **The Game Engine:** This component handles things like setting up the deck dealing cards and keeping track of bets. The engine makes sure these rules work correctly so the agent can work in an environment. The component handles low-level details. It makes the agents job easier, by taking care of the basics. It is the backbone of the project.
- **The Poker Agent:** This is like the brain of the code. It talks to the engine to get an idea of whats happening in the game now. Then it thinks about what to do and makes a smart move. This way it can get better at making decisions over time. The core rules of the game stay the same.

3 State Representation and Feature Quantization

To help the agent make decisions we need to take the basic game situation, which is made up of playing cards and turn it into numbers that the agent can understand. The agent needs to see the game in a way so we have to change the cards into a set of numbers that mean something. This way the agent can look at the game state, which's the cards and make smart choices.

3.1 Hand Strength Quantization

The agent evaluates the state by mapping a 7-card combination to a singular integer value. Let C represent the set of all cards. The evaluator function $f : C^7 \rightarrow \mathbb{R}$ maps the hand to a discrete score based on the following hierarchy:

- **High Card:** $Score = Rank_{max}$
- **Pairs and Sets:** $Score \in \{100, 200, 300, 600, 700\}$ based on rank frequency counts.
- **Sequence and Suit:** $Score \in \{400, 500, 800\}$ for straights and flushes.

This quantization maps the complexity of poker hands into a one-dimensional space, allowing the agent to perform direct numerical comparisons.

4 Mathematical Heuristics Architecture

The agent's decision architecture is built upon the comparison of two normalized variables:

1. **Win Probability (ω):** Derived from the evaluator score normalized against the maximum possible score ($S_{max} = 800$): $\omega = \min(\frac{Score}{800}, 1.0)$.
2. **Pot Odds (ρ):** Calculated as the ratio of the call amount to the total potential pot size:
$$\rho = \frac{AmountToCall}{PotSize + AmountToCall}.$$

The agent acts if and only if $\omega > \rho$. This heuristic architecture ensures that the agent's actions are bounded by the principle of Expected Value (EV), providing a mathematically grounded decision-making process.

5 Reflections on Performance

The agent uses a rule-based approach based on the Expected Value principle. This approach was chosen over complex methods like Monte Carlo simulations or Reinforcement Learning. The reason is that it is fast and easy to understand. The agent's decision-making process is clear. Can be checked mathematically. The agent is strong. Follows all the rules of the game. It also has a safety-check system that makes sure every move is valid. This prevents the agent from making mistakes. Provides a solid base for a competitive player in a real tournament. The agent's design helps it make decisions quickly. It is a choice, for a real-world competition.